

CommonsMesh — NGI Zero Commons Fund Application Attachment

Supplementary Technical Proposal and Milestone Plan

Submitted as an attachment to the NGI Zero Commons Fund application.

Part A: Strategic Alignment with the Next Generation Internet Vision

CommonsMesh is designed as a direct embodiment of the NGI vision: a **human-centric, open, trustworthy, and resilient internet** that empowers individuals and communities rather than surveilling and monetizing them. The European Commission’s Next Generation Internet initiative calls for technology that reflects “openness, diversity and inclusion” and that “enables human potential, mobility and creativity at the largest possible scale.”

CommonsMesh addresses this call in three concrete ways. First, it eliminates structural dependence on corporate platforms for community coordination. Today, the most vulnerable communities—those with the greatest need for mutual aid—are the most dependent on surveillance-based platforms like WhatsApp and Facebook Groups, which monetize their social graphs and expose them to algorithmic manipulation. CommonsMesh provides a genuine alternative: a **local-first, privacy-preserving coordination layer** where all data is stored on the user’s own device and no central entity has access to the community graph.

Second, CommonsMesh advances the NGI goal of **P2P infrastructure as a public good**. By building on libp2p and contributing a well-documented, open protocol for structured community state synchronization, this project creates reusable building blocks for the decentralized internet. The protocol specifications, cryptographic event envelope design, and anti-Sybil mechanisms will be published as open standards that other projects can adopt.

Third, CommonsMesh demonstrates a responsible and privacy-preserving application of **on-device AI**. Rather than sending community data to cloud AI services, the embedded LLM operates entirely locally. This is a concrete contribution to the NGI goal of “intelligent mediators” that serve users rather than extracting value from them.

Part B: Detailed Technical Architecture

B.1 Protocol Layer

The protocol layer is the foundation of CommonsMesh. It defines the structure of all community state changes as **signed event envelopes**. Each event carries a cryptographic signature (Ed25519), a SHA-256 hash of its canonical JSON payload, and a `chain_hash` that links it to the previous event in the user’s personal event log. This creates a tamper-evident, append-only log for every participant.

The event schema supports nine event types, each representing a distinct community action:

Event Type	Description
<code>graph.delta</code>	Declares a need, resource, or skill (the core building block)
<code>project.motion</code>	Proposes a community project or collective action
<code>task.commit</code>	Accepts a task assignment within a project
<code>task.done</code>	Marks a task as completed
<code>capability.grant</code>	Delegates a scoped authorization token to another user
<code>capability.revoke</code>	Revokes a previously granted capability token
<code>vote.cast</code>	Casts a vote on a community motion
<code>dispute.report</code>	Reports a violation or broken commitment
<code>trust.attest</code>	Provides a social proof for another user’s trustworthiness

The validator pipeline processes incoming events through seven sequential checks: schema validation, signature verification, payload hash integrity, temporal window (± 5 minutes), replay prevention (TTL-bounded nonce cache), chain link integrity, and authorization policy. An event that fails any check is silently dropped and the sender's trust score is penalized.

B.2 Network Layer (libp2p)

The network layer uses libp2p to create a fully serverless P2P mesh. The node factory configures TCP and WebSocket transports, mDNS for local peer discovery, Kademlia DHT for wide-area peer discovery, and GossipSub for topic-based message propagation. Community topics are namespaced as `commonsmesh/v1/{communityId}`, allowing geographically distinct communities to operate in isolated namespaces.

The incremental sync protocol is the most technically novel component of the network layer. When two nodes connect, they exchange Bloom filters representing their known event hashes. The requesting node identifies missing events from the difference and requests them in chunks. This approach minimizes bandwidth consumption—a critical constraint for users in areas with limited mobile data—while ensuring eventual consistency of the community graph.

B.3 Local Graph Database

The local graph database is implemented using SQLite (via `better-sqlite3` on desktop and `expo-sqlite` on mobile). The schema stores events in an append-only `events` table and maintains a derived `graph_nodes` and `graph_edges` table that represents the current state of the community graph. A `nonces` table provides persistent replay protection that survives app restarts.

The graph store supports queries for active needs, available resources, and user profiles, which feed directly into the LLM matching engine. All data is stored exclusively on the user's device; the network layer only propagates signed incremental events, never the full graph.

B.4 On-Device LLM Integration

The LLM integration layer uses `llama.rn` (a React Native binding for `llama.cpp`) to run GGUF-format models locally. The matching engine operates in two stages. In the

first stage, a deterministic rule engine performs fast, exact matching (e.g., a user who needs “a ladder” and a user who has “a ladder available”). In the second stage, the LLM is invoked for semantic matching (e.g., a user who needs “help moving furniture” and a user who “has a van and free time on weekends”) and for detecting aggregate patterns (e.g., five users who each need “a community garden plot” could collectively negotiate with a landowner).

The LLM prompt is carefully structured to include a JSON summary of the local graph, a list of active needs and resources, and a request for actionable suggestions. The model is instructed to output structured JSON, which is then parsed and displayed as actionable “motions” in the app UI.

Part C: Milestone Plan

The project is structured as a 9-month development cycle, divided into five milestones. Each milestone produces a publicly available, verifiable deliverable.

Milestone	Duration	Deliverables	Verification
M1: Protocol Hardening	Months 1–2	Finalized event schema (JSON Schema), complete validator pipeline with anti-Sybil mechanisms, capability token system, test suite with $\geq 90\%$ coverage	Published to GitHub; test results in CI
M2: Network Layer	Months 3–4	libp2p node factory, Bloom filter sync protocol, integration tests with 3+ simulated nodes	Published to GitHub; demo video
M3: LLM Integration	Months 5–6	llama.rn integration, two-stage matching engine, benchmark report (inference time, battery usage on mid-range Android device)	Published to GitHub; benchmark report
M4: Mobile App	Months 7–8	Complete React Native (Expo) app with all 9 screens, keychain key management, end-to-end test with 2 physical devices	Published to GitHub; demo video
M5: Pilot & Documentation	Month 9	Pilot deployment report (≥ 10 users), comprehensive technical documentation, user guide, security audit preparation checklist	Published to GitHub; pilot report

Part D: Sustainability and Long-Term Impact

CommonsMesh is designed for long-term sustainability as a digital commons. The AGPL-3.0 license ensures that any commercial use of the codebase must contribute improvements back to the community. The local-first architecture means there are no server costs to sustain—users’ own devices form the network.

After the initial grant period, sustainability will be pursued through three channels. First, the project will apply for follow-on NGI Zero funding to scale the protocol and add features such as offline mesh networking (Bluetooth/Wi-Fi Direct) and integration with community currencies. Second, the open protocol specification will be submitted to relevant standards bodies to encourage adoption by other community technology projects. Third, community-operated relay nodes (for users without persistent internet

access) can be funded through voluntary contributions from the communities that benefit from the network.

The most significant long-term impact of CommonsMesh is not the app itself, but the **open protocol for structured community coordination**. Just as ActivityPub became the foundation for the Fediverse, the CommonsMesh protocol can become a reusable building block for any application that needs to coordinate decentralized, privacy-preserving community action.

Part E: Relation to Existing NGI Zero Projects

CommonsMesh is complementary to several existing NGI Zero efforts. It builds on the libp2p ecosystem that has received NGI support, and its local-first data model is inspired by the Solid project's vision of user-owned data. Unlike Solid (which requires a personal data pod server), CommonsMesh is fully serverless. Unlike Secure Scuttlebutt (which is optimized for social feeds), CommonsMesh is optimized for structured, actionable coordination. CommonsMesh does not compete with federated social networks; it fills a distinct gap as a **coordination layer for real-world collective action**.

We are open to collaboration with any NGI Zero-funded projects working on decentralized identity, local-first data, or P2P networking, and welcome suggestions from the review team in this regard.

Part F: Open Source Commitment

All software produced under this grant will be released under the **AGPL-3.0-only** license. This includes:

- The complete protocol specification and JSON Schema
- The TypeScript implementation of the protocol engine, network layer, and database layer
- The React Native mobile application
- All documentation, including technical specifications, user guides, and the pilot deployment report

- All benchmark data and test suites

The repository is publicly available at: <https://github.com/e982happy/CommonsMesh>

We commit to maintaining the repository as an active open-source project, responding to issues and pull requests, and publishing all grant-funded work incrementally as it is completed.